ECEN 5113

HW: 3

Saira Khan

October 6, 2025

Procedure:

The first part of this lab involved creating an FSM using two different models of Verilog. The circuit takes a one-bit input, X, which represents a four-bit binary number (N). The four-bit binary is added to three, and a one-bit output, S, represents that four-bit binary number (N+3). A one-bit output, V, represents the case where S overflows. S can represent the numbers 0-15, so if there is a case where S is greater than 15, V=1. If S is equal to or less than 15, V =0. The following state diagram shows the logic of the FSM with the inputs/outputs being represented as X/(S, V).

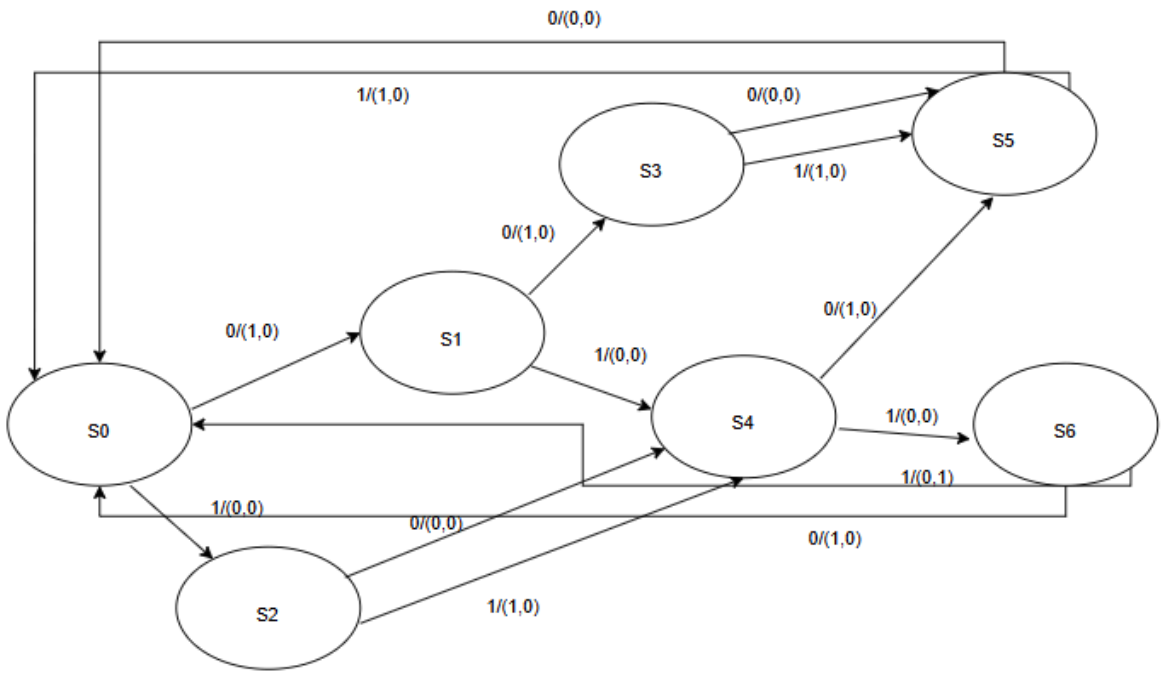

Figure 1: State Diagram of the FSM

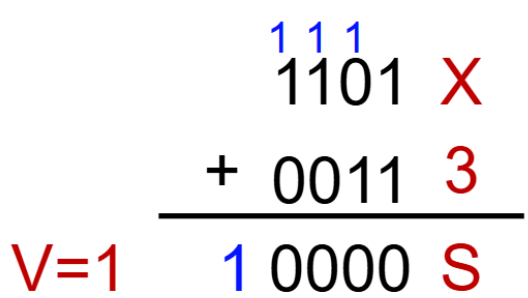

Figure 2: Example of Addition happening in FSM

The above figure shows an example of what this finite state machine is doing. If N equals 1101 (decimal value 13), the FSM should work like this:

Start in S0 (state 0)

Input starts from the far right (X=1)

X=1 | State = S2 | Output = (0,0)

X = 0 | State = S4 | Output = (0,0)

X = 1 | State = S6 | Output = (0,0)

X = 1| State = S0 | Output = (0,1)

Starting from the far right of the binary number 1101, the first input (X) will be 1. Looking at the state diagram, an input of X = 1 means a transition to State 2, and the outputs S and V are 0,0, respectively. Going from right to left, X will now be 0, and the next state is S4. This goes on till all four inputs of X are executed. The output for S each time is 0, so N+3 = 0000, and V equals 1 (carry over of 1). This makes sense, looking at the binary addition above to check the results, as well as adding 13+3 = 16 and seeing that 16>15 (so V has to be 1).

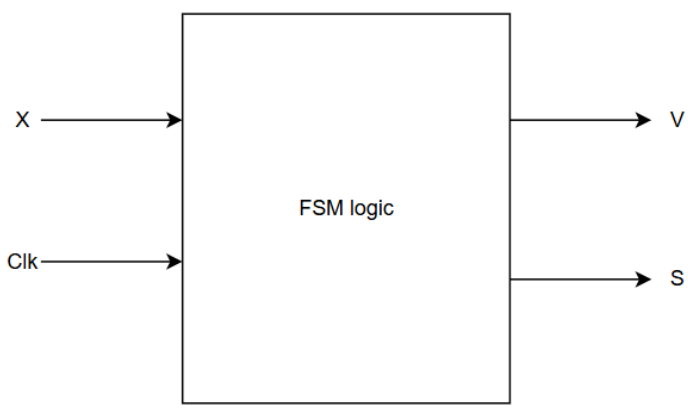

Figure 3: FSM Block Diagram

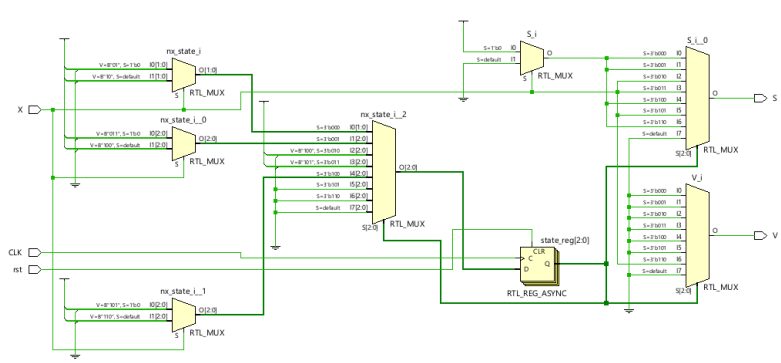

Figure 4: FSM Circuit Diagram

To create the behavioral model of this logic in Verilog, first, a module is defined with the inputs: X, CLK, and rst, and outputs: S and V. Variables are created to hold the value of the current and next state as well. Parameters are created to hold the values of each state so that the states can be referenced by their name. Parameters are a feature used to define constants in a module, and allow those constants to be reused, and these values are fixed. An always statement is used for the sequential logic of the circuit. So on every positive edge of the clock, the next state value is copied into the state value (so that it becomes the current value). If reset is enabled, then the state becomes S0 (default starting). Another always block is used for the combinational logic of the circuit. This part computes the next state and outputs based on the current inputs. Case statements are made based on the table provided. A testbench is then created to test the input X = 1011 1100 1101. An initial block is created to create the logic for the clock. A forever statement is made that switches the clock every 5ns from low (0) to high (1), totaling a 10ns period. The reset logic is initialized so that the FSM starts in a known state. The reset is asserted at 0ns, so at 5ns (the first rising edge of the clock), the FSM resets to a known state (S0). The clock falls at 10ns, and at 12ns (between 10 and 15), the reset is deasserted. This is done between clock cycles so that race conditions don't arise. A last initial block is created to test the different inputs of X. X is changed every ¼ clock period after the rising edge. The first rising edge starts at 5ns, so X changes at 7.5 (2.5 + 5). Then every 10ns after that, X is changed. The next change will then happen at 17.5ns (5ns for the falling edge, and another 5ns for the rising edge).

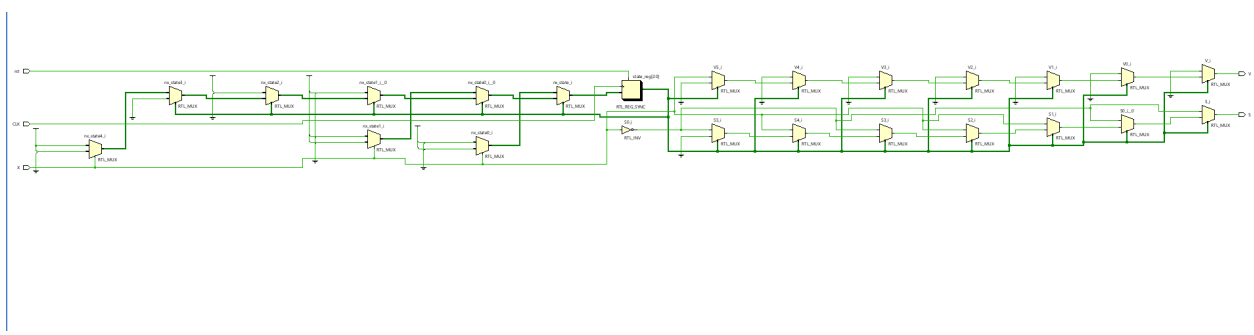

Figure 5: Dataflow FSM Diagram

Table 1: FSM State Logic

Present State	Next State		Output	
	X=0	X=1	X=0	X=1
S0	S1	S2	1, 0	0, 0
S1	S3	S4	1, 0	0, 0
S2	S4	S4	0, 0	1, 0
S3	S5	S5	0, 0	1, 0
S4	S5	S6	1, 0	0, 0
S5	S0	S0	0, 0	1, 0
S6	S0	S0	1, 0	0, 1

The second part of this lab involves creating a BCD counter. The BCD counter is a 4-bit counter that counts up/down from 0 to 9. The state changes happen on the rising edge of the clock, except for the clr function, which is asynchronous and resets the counter. The three-bit input D holds the value that the counter will start at, and Q is the three-bit output, which will either increment/decrement based on the value of UP (0/1). Once the count, Q, reaches 9, it wraps back around to 0. To model this logic in Verilog, an always block is created with if statements to match the logic of the circuit: if enable equals 1, if load equals 1, if up equals and vice versa. A test bench is set up to test the logic. Each clock period is set up to be 10ns, and the state changes on the rising edge of the clock. To create a two-digit decimal counter, two BCD counters are instantiated. One is used for the least significant bit (ones place) and the other is used for the most significant bit (tens place). The most significant bit is enabled when the carry out of the least significant bit is 1 or the least significant bit is at 9 or 0. A test bench is again created to test the logic.

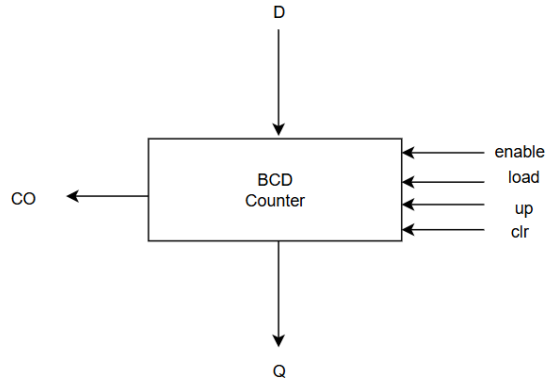

Figure 6: BCD Counter Block Diagram

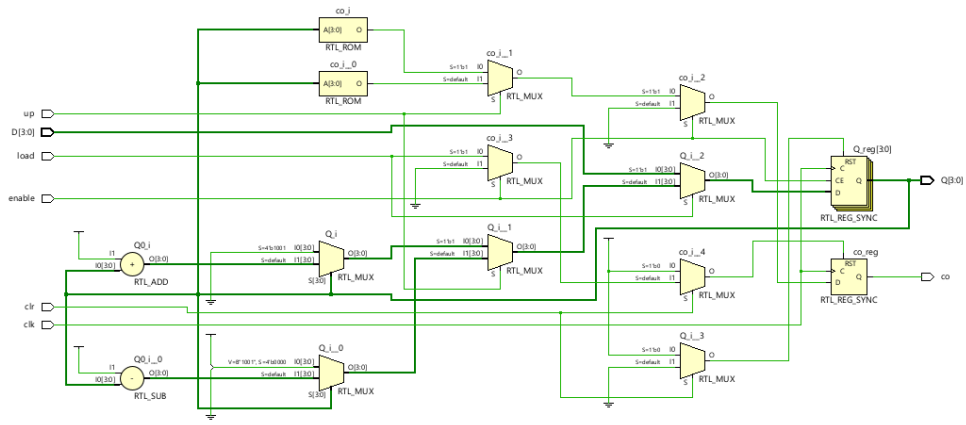

Figure 7: BCD Counter One-Digit Circuit Diagram

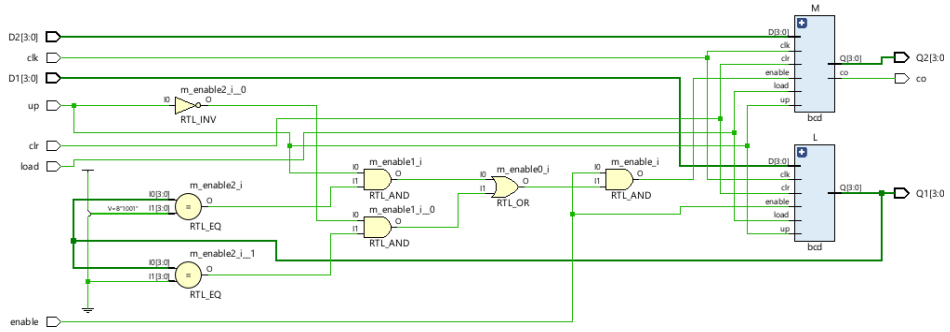

Figure 8: BCD Counter Two-Digit Circuit Diagram

Observations:

Through this lab, I once again learned the importance of timing. The first time I simulated the FSM behavioral code, I was not getting the right answer, but I reviewed my logic multiple times, and it seemed right. I then realized, in my testbench, I had changed the input of X and then waited a whole clock period before displaying the results, so everything was off. Changing the delay between when the input was changed and when the results were displayed fixed the problem. This lab showed me the importance of initializing variables as well. Simulating the code and looking at the waveform, there was a red block that showed up for the first time, and the value showed up as X. I realized that this was because my variables were not initialized, so they started out as unknowns. Initializing my variables helped solve the issue of the red block appearing on my waveforms. The waveforms for part 2 of the lab do have some red in them due to the outputs. The outputs are wires and can not be initialized in the testbench, so until the first clock cycle occurs (and the always block executes), the values are marked as x's (unknowns). Once the first clock cycle occurs, however, the values initialize themselves.

Results:

FSM Behavioral:

Figures 9 and 10 show the output of the behavioral model of the FSM circuit. Looking at Figure 9, the input X starts at the far-right (LSB) of the binary number 13 (1101). S shows the right output as each value is 0, and the last output for V shows there is a carry out. This is also verified in Figure 2.

```
time=8000 | X=1 | S=0 | V=0
time=18000 | X=0 | S=0 | V=0
time=28000 | X=1 | S=0 | V=0
time=38000 | X=1 | S=0 | V=1
time=48000 | X=0 | S=1 | V=0
time=58000 | X=0 | S=1 | V=0
time=68000 | X=1 | S=1 | V=0
time=78000 | X=1 | S=1 | V=0
time=88000 | X=1 | S=0 | V=0
time=99000 | X=1 | S=1 | V=0
time=109000 | X=0 | S=1 | V=0
time=119000 | X=1 | S=1 | V=0
```

Figure 9: FSM Behavioral Output

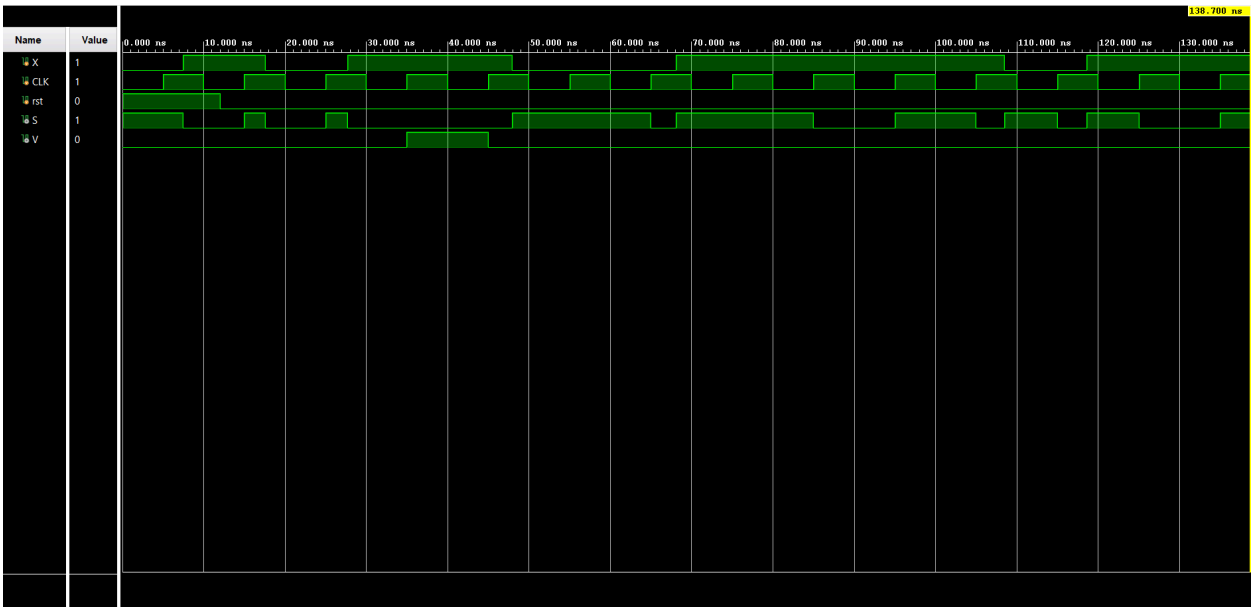

Figure 10: FSM Behavioral Waveform

Figure 10 shows the waveform of the circuit. It can be seen that each clock cycle is 10 ns: 5ns for low, and 5ns for high. It can be seen that X changes ¼ of the way through the clock period, which is halfway into the rising edge of the clock cycle. V equals 1 can be seen on the waveform as well.

<u>FSM Dataflow:</u>

Figures 11 and 12 show the output for the dataflow model of the FSM. The output is exactly the same as the output of the behavioral model of the FSM, which is correct. The same testbench was used to test the logic.

```
# run 1000ns
time=8000 | X=1 | S=0 | V=0
time=18000 | X=0 | S=0 | V=0
time=28000 | X=1 | S=0 | V=0
time=38000 | X=1 | S=0 | V=1
time=48000 | X=0 | S=1 | V=0
time=58000 | X=0 | S=1 | V=0
time=68000 | X=1 | S=1 | V=0
time=78000 | X=1 | S=1 | V=0
time=88000 | X=1 | S=0 | V=0
time=99000 | X=1 | S=1 | V=0
time=109000 | X=0 | S=1 | V=0
time=119000 | X=1 | S=1 | V=0
```

Figure 11: FSM Dataflow Output

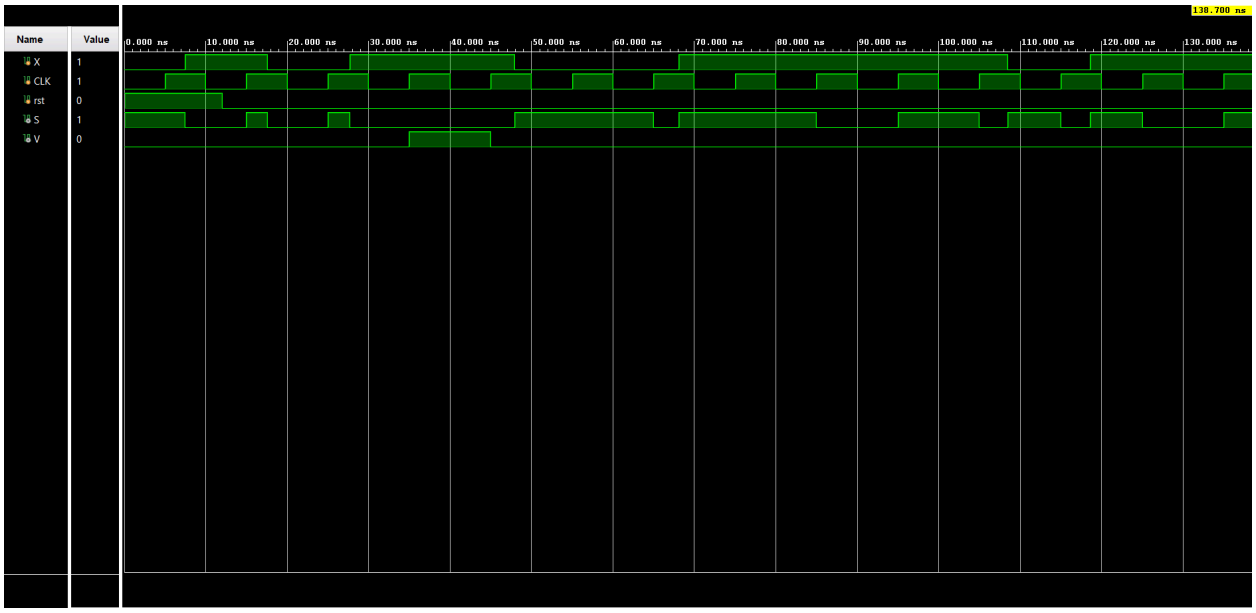

Figure 12: FSM Dataflow Waveform

<u>BCD One-Digit:</u>

```
time=20000 | D=0110 | Q=0110 | co=0
time=30000 | D=0110 | Q=0111 | co=0
time=40000 | D=0110 | Q=1000 | co=0
time=50000 | D=0110 | Q=1001 | co=0
time=60000 | D=0110 | Q=0000 | co=1
time=70000 | D=0110 | Q=1001 | co=1
time=80000 | D=0110 | Q=0000 | co=0
```

Figure 13: BCD One-Digit Counter Output

Figures 13 and 14 show the output for the BCD One-Digit circuit. D starts at 6 and once the load is set to high (1), Q takes the value of D (at 20ns). The count then increments four times: 7, 8, 9, 0 (restarts). The count is decremented back to 9, showing the wrap-around logic works. Then the counter is cleared (Q = 0). The waveform confirms this output by showing the value of Q changing correctly. The clock can be seen to have a 10ns clock period. When the clock goes high, the state changes.

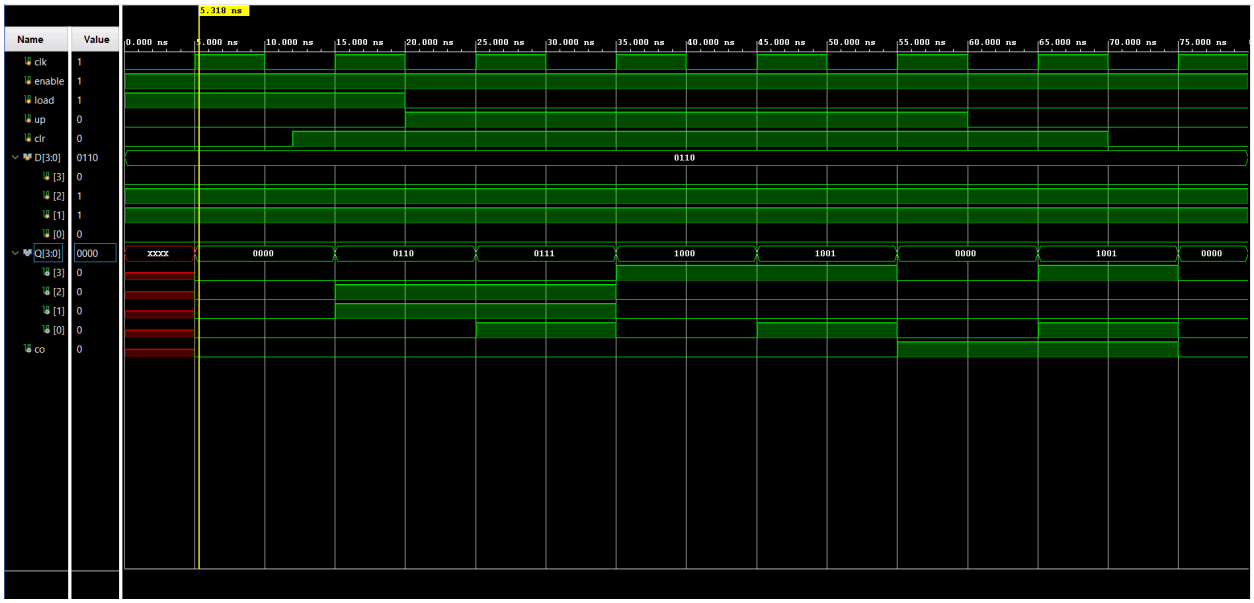

Figure 14: BCD One-Digit Counter Waveform

<u>BCD Two-Digit:</u>

```
time=20000 | D1=0111 | D2=1001 | Q1=0111 | Q2=1001 |co=0
time=30000 | D1=0111 | D2=1001 | Q1=1000 | Q2=1001 |co=0
time=40000 | D1=0111 | D2=1001 | Q1=1001 | Q2=1001 |co=0
time=50000 | D1=0111 | D2=1001 | Q1=0000 | Q2=0000 |co=1
time=60000 | D1=0111 | D2=1001 | Q1=0001 | Q2=0000 |co=0
time=70000 | D1=0111 | D2=1001 | Q1=0010 | Q2=0000 |co=0
time=80000 | D1=0111 | D2=1001 | Q1=0010 | Q2=0000 |co=0
time=90000 | D1=0111 | D2=1001 | Q1=0010 | Q2=0000 |co=0
time=100000 | D1=0111 | D2=1001 | Q1=0001 | Q2=0000 |co=0
time=110000 | D1=0111 | D2=1001 | Q1=0000 | Q2=0000 |co=0
time=120000 | D1=0111 | D2=1001 | Q1=1001 | Q2=1001 |co=1
time=130000 | D1=0111 | D2=1001 | Q1=1000 | Q2=1001 |co=0
time=140000 | D1=0111 | D2=1001 | Q1=0000 | Q2=0000 |co=0
```

Figure 15: BCD Two-Digit Counter Output

Figures 15 and 16 show the output for the BCD Two-Digit output. Q1 shows the ones place changing, and Q2 shows the tens place changing. At the start, Q1 is equal to 7 and Q1 is equal to 9 (97). This then increments five times, which can be seen in the Q1 column. Once the value reaches 100, the output is reset to 0, and the carry out shows 1. Then the enable is set to 0, so for two clock cycles, nothing happens. The enable is set to 1 again, and the count is decremented four times. The change from 0 back to 99 can be seen at time 120000ps. The counter is then cleared. The waveform confirms this behavior again. The waveform shows the clock switching

every 5ns (10ns time period). States are changed at the rising edge of the clock, which can be seen at the 5ns mark, where Q1, Q2, and co (the outputs) change from undefined to 0.

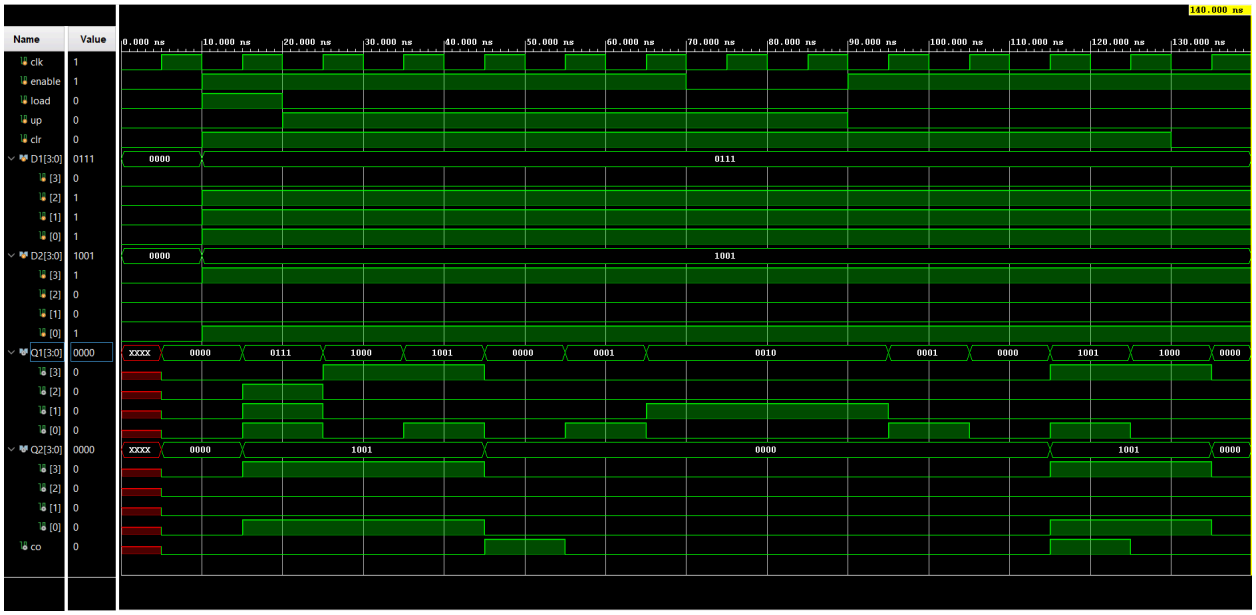

Figure 16: BCD Two-Digit Counter Waveform

Conclusion:

During this lab, an FSM circuit was created using two different models of Verilog. Though the logic was the same, the structure of the actual code was different. Case statements were used to model the different states in the behavioral model, while continuous statements were used in the dataflow model. A one-digit BCD counter was then created and instantiated to create a two-digit BCD, where one BCD counter represented the ones place and the second represented the tens place. Timing was shown to be important, once again, when it came to when inputs were changed and results were displayed. Initializing inputs also showed to be important for the creation of waveforms.

Appendix:

```
//fsm behavior module


//define module
module fsm(
   //define inputs/outputs
   input X,
```

```
    input CLK,
    input rst,
    output reg S,
    output reg V


);

    reg [2:0] state; //reg - assigning value to signal inside always block
    reg [2:0] nx_state; //next state


    //parameters that hold values of each state
    //states can be referenced by name
    parameter S0 = 3'b000;
    parameter S1 = 3'b001;
    parameter S2 = 3'b010;
    parameter S3 = 3'b011;
    parameter S4 = 3'b100;
    parameter S5 = 3'b101;
    parameter S6 = 3'b110;




    //always statement
    //change on the positive edge of clock (rising edge)
    //sequential logic - reprsents flip flops that store state
    // @ every rising edge of clock -- it copies next state into state
    always @(posedge CLK or posedge rst)
        begin
            if (rst)
            state <= S0;
            else
            state <= nx_state; //update state on clock edge
```

```
    end

    //combinational logic -- computes next state and outputs from current state
always @(*)
    begin

        //default
        S = 1'b0;
        V = 1'b0;
        nx_state = state;

        case(state)
            S0:
                begin
                if(X==0)
                    begin
                        nx_state = S1;
                        S = 1'b1;
                        V = 1'b0;
                    end
                else
                    begin
                        nx_state = S2;
                        S = 1'b0;
                        V = 1'b0;
                    end
            end



            S1:
                begin
                    if(X==0)
                        begin
                            nx_state = S3;
                            S = 1'b1;
```

```
            V = 1'b0;
        end
    else
        begin
            nx_state = S4;
            S = 1'b0;
            V = 1'b0;
        end
end

S2:
    begin
    if(X==0)
        begin
            nx_state = S4;
            S = 1'b0;
            V = 1'b0;
        end
        else
        begin
            nx_state = S4;
            S = 1'b1;
            V = 1'b0;
        end
    end

S3:
    begin
        if(X==0)
            begin
                nx_state = S5;
                S = 1'b0;
                V = 1'b0;
            end
        else
```

```
        begin
            nx_state = S5;
            S = 1'b1;
            V = 1'b0;
        end
    end

S4:
    begin
        if(X==0)
            begin
                nx_state = S5;
                S = 1'b1;
                V = 1'b0;
            end
        else
            begin
                nx_state = S6;
                S = 1'b0;
                V = 1'b0;
    end
    end

S5:
    begin
    if(X==0)
        begin
            nx_state = S0;
            S = 1'b0;
            V = 1'b0;
        end
    else
        begin
            nx_state = S0;
            S = 1'b1;
```

```
                V = 1'b0;
            end
        end

    S6:
        begin
            if(X==0)
                begin
                    nx_state = S0;
                    S = 1'b1;
                    V = 1'b0;
                end
            else
                begin
                    nx_state = S0;
                    S = 1'b0;
                    V = 1'b1;
                end
        end

    default:
        begin
            nx_state = S0;
            S = 1'b0;
            V = 1'b0;
        end
    endcase

end


endmodule
```

```
//testbench
//define module
module fsm_tb;
   //declare inputs/outputs
   reg X;
   reg CLK;
   reg rst;
   wire S;
   wire V;

   //instantiate moudle
   fsm d(X, CLK, rst, S, V);

   initial
      begin
         CLK = 0;
         //forever - run continously
         // full period (0 -- 1 -- 0)
         forever #5 CLK = ~CLK; //toggle every 5 ns - period 10ns
      end

   //reset
   initial
      // assert reset so FSM doesnt start in random state or undefined
      begin
         rst = 1;
         #12;
         rst = 0;
      end


   //initial block -- test
   initial
      begin
         //initalize input
```

```
X = 0;
#7.5; //wait 2.5ns after first posedge (5ns)




//clock perdio = 10ns
//1/4 period = 10ns/4 = 2.5 ns

//test case X = 1011 1100 1101

X = 1;
#0.1;
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
#10; //add delay - one full clcok period later

X = 0;
#0.1;
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
#10; //add delay

X = 1;
#0.1;
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
#10; //add delay

X = 1;
#0.1;
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
#10; //add delay

X = 0;
```

```
#0.1;
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
 #10; //add delay

X = 0;
#0.1; //add delay
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
 #10; //add delay

X = 1;
#0.1; //add delay
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
 #10; //add delay
X = 1;
#0.1; //add delay
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
#10; //add delay
X = 1;
#0.1; //add delay
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);
 #10; //add delay
X = 1;
#0.1; //add delay
$display("time=%0t | X=%b | S=%b | V=%b",
 $time, X, S, V);

 #10; //add delay


X = 0;
```

```
            #0.1; //add delay
            $display("time=%0t | X=%b | S=%b | V=%b",
             $time, X, S, V);
             #10; //add delay
            X = 1;
            #0.1; //add delay
            $display("time=%0t | X=%b | S=%b | V=%b",
             $time, X, S, V);



            #20;


            $finish; //finish simulation
        end
endmodule
//fsm dataflow module


//define module
module fsm(
    //define inputs/outputs
    input X,
    input CLK,
    input rst,
    output  S,
    output  V



    );









    //parameters that hold values of each state
    //states can be referenced by name
```

```
parameter S0 = 3'b000;
parameter S1 = 3'b010;
parameter S2 = 3'b001;
parameter S3 = 3'b101;
parameter S4 = 3'b011;
parameter S5 = 3'b100;
parameter S6 = 3'b111;

reg [2:0] state = S0; //wire - continious statements (assign)
wire [2:0] nx_state; //next state



assign nx_state =
(state == S0) ? (X ? S2: S1): // if state = S) and X = 1 then next state = S2 else S1
(state == S1) ? (X ? S4: S3):
(state == S2) ? S4:
(state == S3) ? S5:
(state == S4) ? (X ? S6 : S5):
(state == S5) ? S0:
(state == S6) ? S0:
S0; //default



assign S =
(state == S0) ? (~X):
(state == S1) ? (~X):
(state == S2) ? (X):
(state == S3) ? (X):
(state == S4) ? (~X):
(state == S5) ? (X):
(state == S6) ? (~X):
1'b0; //default

assign V=
```

```
    (state == S0) ? 1'b0:
    (state == S1) ? 1'b0:
    (state == S2) ? 1'b0:
    (state == S3) ? 1'b0:
    (state == S4) ? 1'b0:
    (state == S5) ? 1'b0:
    (state == S6) ? (X ? 1'b1 : 1'b0):
    1'b0; //default

    //reset logic/clock logic
    always @(posedge CLK)
        begin
            if(rst)
                state <= S0;
            else
                state <=nx_state;
            end
endmodule

//BCD counter

//define module
module bcd(
    input clk,
    input enable,
    input load,
    input up,
    input clr,
    input [3:0] D,
    output reg [3:0] Q,
    output reg co



    );
```

```
always @(posedge clk)
  begin
    if(!clr)
    begin
      Q <= 4'b0000;
      co <= 1'b0;

    end
    else
      begin
      co <=1'b0;
      if(enable)
        begin
        if(load)
          begin
            Q<=D;
          end
        else
          begin
            if(up)
              begin
              //increment
                if(Q==4'b1001) //wrap around logic
                  begin
                    co <=1'b1;
                    Q <=4'b0000;
                  end
                else
                  begin
                    Q<= Q +4'b0001;
                  end
               end
            else
              begin
```

```
                    if(Q==4'b0000)
                        begin
                            co <=1'b1;
                            Q<=4'b1001;
                        end
                    else
                        begin
                            Q<=Q-4'b0001;
                        end
                end
            end
        end
    end
    end

endmodule


//testbench
//define module
module bcd_tb;
    //define inputs/outputs
    reg clk;
    reg enable;
    reg load;
    reg up;
    reg clr;
    reg [3:0] D;
    wire [3:0] Q;
    wire co;
```

```
//instantiate module
bcd d(clk, enable, load, up, clr, D, Q, co);



//set up clock
initial
   begin
      clk = 0;
      forever #5 clk = ~clk; //clock period of 10ns
   end


//assert clr/"rst" - ensure Q start at 0
initial
   begin
      clr = 0; //clr = 0 --counter is reset
      #12;
      clr =1;
   end


//test inputs
initial
   begin
      enable = 0;
      load = 0;
      up = 0;
      clr = 0;
      D = 4'b0000;



      //load counter w 6
      enable = 1;
```

```
load = 1;
D = 4'b0110;
#20; //add delay
$display("time=%0t | D=%b | Q=%b | co=%b",
 $time, D, Q, co);

//increment four times - keep UP = 1 for four clock cycles
//40ns
load = 0;
up = 1;
#10; //add delay
$display("time=%0t | D=%b | Q=%b | co=%b",
 $time, D, Q, co);
#10; //add delay
$display("time=%0t | D=%b | Q=%b | co=%b",
 $time, D, Q, co);
#10; //add delay
$display("time=%0t | D=%b | Q=%b | co=%b",
 $time, D, Q, co);

#10; //add delay
$display("time=%0t | D=%b | Q=%b | co=%b",
 $time, D, Q, co);

//decrement once - one clock cycle
up= 0;
#10; //add delay
$display("time=%0t | D=%b | Q=%b | co=%b",
 $time, D, Q, co);



//clear counter
clr = 0;
#10; //add delay
$display("time=%0t | D=%b | Q=%b | co=%b",
```

```
            $time, D, Q, co);


          $finish; //finish simulating
        end


endmodule
//BCD counter

//define module
module bcd(
    input clk,
    input enable,
    input load,
    input up,
    input clr,
    input [3:0] D,
    output reg [3:0] Q,
    output reg co


    );


    always @(posedge clk)
        begin
          if(!clr)
          begin
              Q <= 4'b0000;
              co <= 1'b0;

          end
          else
              begin
```

```
co <=1'b0;
if(enable)
   begin
   if(load)
      begin
         Q<=D;
      end
   else
      begin
         if(up)
            begin
            //increment
               if(Q==4'b1001) //wrap around logic
                  begin
                     co <=1'b1;
                     Q <=4'b0000;
                  end
               else
                  begin
                     Q<= Q +4'b0001;
                  end
              end
         else
            begin
               if(Q==4'b0000)
                  begin
                     co <=1'b1;
                     Q<=4'b1001;
                  end
               else
                  begin
                     Q<=Q-4'b0001;
                  end
         end
   end
```

```
            end
        end
    end


endmodule


//bcd two digit
//define module
module bcd_two(
    input clk,
    input enable,
    input load,
    input up,
    input clr,
    input [3:0] D1,
    input [3:0] D2,
    output [3:0] Q1,
    output [3:0] Q2,
    output co
    );


    //internal wires
    wire co_lsb;


    //LSB - ones place
```

```
    bcd L(clk, enable, load, up, clr, D1, Q1, co_lsb);



    wire m_enable;
    //enable msb (10s place) when lsb carries out (10..)
    assign m_enable = enable && ((up && Q1 == 4'b1001) || (!up && Q1==4'b0000));

    //MSB - 10s place
    bcd M(clk, m_enable, load, up, clr, D2, Q2, co);









endmodule



//testbench
//define module
module bcd_two_tb;
    //define inputs/outputs
    reg clk;
    reg enable;
    reg load;
    reg up;
    reg clr;
    reg [3:0] D1;
    reg [3:0] D2;
    wire [3:0] Q1;
    wire [3:0] Q2;
    wire co;



    //instantitate module
```

```
bcd_two d(clk, enable, load, up, clr, D1, D2, Q1, Q2, co);



//set up clock
initial
   begin
      clk = 0;
      forever #5 clk = ~clk; //clock period of 10ns
   end



//test cases
initial
   begin



   //initalize stuff
   clr = 0;
   enable = 0;
   load = 0;
   up = 0;
   D1 = 4'b0000;
   D2 = 4'b0000;
   #10;
   clr = 1;





   //load counter w 97
   enable = 1;
   load = 1;
   D1 = 4'b0111;
   D2 = 4'b1001;
   #10; //add delay
```

```
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);

//increment counter 5 times
//clock cyles 5 - 50 ns
load = 0;
up = 1;

#10; //add delay
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);
#10; //add delay
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);
#10; //add delay
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);
#10; //add delay
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);
#10; //add delay
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);

//do nothing for two clock periods
enable = 0;
#10; //add delay
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);

#10; //add delay
$display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
 $time, D1,D2, Q1, Q2, co);
```

```
        //decrement four times
        //up = 0
        //four clock cycles - 40 ns
        enable = 1;
        up = 0;
        #10; //add delay
        $display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
         $time, D1,D2, Q1, Q2, co);

        #10; //add delay
        $display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
         $time, D1,D2, Q1, Q2, co);

        #10; //add delay
        $display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
         $time, D1,D2, Q1, Q2, co);

        #10; //add delay
        $display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
         $time, D1,D2, Q1, Q2, co);

        //clear counter
        clr = 0;
        #10; //add delay
        $display("time=%0t | D1=%b | D2=%b | Q1=%b | Q2=%b |co=%b",
         $time, D1,D2, Q1, Q2, co);

        //finish
        $finish;
    end
endmodule
```